

Software Testing

From Traditional Principles to Modern Automation

What is Software Testing?

The process of evaluating and verifying that a software product or application does what it is supposed to do.

The goal is to find defects, ensure quality, and meet business requirements *before* the software is released to the end-user.

Briefing: Why is Testing Critical?

-  **Saves Money:** Finding bugs early is exponentially cheaper than fixing them in production.
-  **Protects Customer Trust:** A buggy or unreliable product erodes user trust and drives customers to competitors.
-  **Preserves Brand Reputation:** Public failures can damage a brand's reputation, impacting sales and partnerships.
-  **Ensures Security & Compliance:** Testing is essential to protect user data and comply with legal standards (e.g., GDPR, PCI).

The Tester's Paradox: Developer vs. QA Goals

The Developer's Goal

To **build** the software and prove that it **works**. Their primary mindset is construction, following the "happy path" to make a feature functional.

The Tester's Goal

To **find bugs** and prove that it **breaks**. Their primary mindset is "constructive destruction," looking for edge cases, failure points, and unexpected user behavior.

The Exponential Cost of Bugs

*A bug found during the **Requirements** phase might cost \$1 to fix.*

*The ***same bug*** found in **Production** could cost \$1,000 to \$10,000 to fix, factoring in downtime, patches, and lost reputation.*

Part 1

The Foundations of Testing

The Levels of Testing



Unit Testing

Testing individual components or blocks of code in isolation.



Integration Testing

Testing how two or more modules interact and work together.



System Testing

Testing the complete, integrated application as a whole.



Acceptance Testing

Validating that the software meets the business requirements.

Core Testing Techniques



Black Box

No knowledge of internal code.
Testing based on requirements.
(e.g., "Does the login button work?")



White Box

Full knowledge of internal code.
Testing code paths and logic. (e.g.,
Unit Tests)



Grey Box

Partial knowledge. Testing with
access to the database or APIs to
set up data. (e.g., API Testing)

Types: Functional vs. Non-Functional

Functional Testing

Answers: **"Does it work?"**

Validates specific actions and functions. (e.g., "Can a user add an item to the cart?").

Non-Functional Testing

Answers: **"How well does it work?"**

Tests attributes like performance, usability, and security. (e.g., "Is the login fast?")

Critical Concept: Regression Testing

The process of re-running tests to ensure that recent code changes have not broken any existing or unrelated functionality. This is the #1 candidate for test automation.

Part 2

Deep Dive on Non-Functional

Performance: Load vs. Stress Testing

Load Testing

Tests the system's performance under **expected peak conditions**. (e.g., "How does the site handle 1,000 users at once?" - our expected Black Friday load).

Stress Testing

Pushes the system **beyond its limits** to find the breaking point and observe its recovery. (e.g., "At what point does the site crash? 5,000? 10,000?").

Deep Dive: Security Testing

-  Uncovers vulnerabilities, threats, and risks to protect data.
-  **Authentication/Authorization:** Can a normal user access an admin page?
-  **SQL Injection (SQLi):** Can a user input a database command into a search box?
-  **Cross-Site Scripting (XSS):** Can a user inject malicious code into a comments section?

Deep Dive: Usability & Acceptance (UAT)

Usability Testing

Performed by testers (or real users) to check if the application is **user-friendly and intuitive**. (e.g., "Is the checkout process confusing?").

User Acceptance Testing (UAT)

Performed by the **client or business stakeholders**. It is the final check to confirm the software meets all business requirements before release.

Part 3

Testing & The Software Lifecycle (SDLC)

SDLC: The Waterfall Model

*In Waterfall, testing is a distinct, separate phase that occurs ***after*** all development is complete.*

***The Problem:** Bugs are found very late, making them expensive to fix.
The testing phase becomes a bottleneck.*

SDLC: The Agile Model

*In Agile, testing is not a phase; it's a continuous activity. Testers are embedded within the development team and test *during* the sprint.*

***The "Shift-Left" Approach:** Testing starts earlier, feedback is constant, and the whole team is responsible for quality.*

SDLC: The Hybrid Model

A mix of models. Often, this involves a Waterfall approach for foundational systems (like a core database) and an Agile approach for user-facing applications (like a mobile app) that are built on top. The testing strategy must be bi-modal to support both.

Testing Strategy Comparison

For Agile

- Heavy test automation (Unit, API, UI).
- Continuous Integration (CI) runs tests on every change.
- Exploratory and UAT at the end of *each* sprint.

For Hybrid

- "Waterfall" part has large-scale System and UAT phases.
- "Agile" part uses rapid, sprint-based automation.
- Complex integration testing is needed where the two models meet.

Part 4

The World of Automation

What is Automation Testing?

-  Using specialized software tools (like Selenium) and scripts to automatically execute tests.
-  It compares actual results with expected results and reports the outcomes.
-  Ideal for tests that are repetitive, high-volume, or difficult to perform manually (e.g., Regression Tests).
-  It is a *complement* to manual testing, not a replacement.

The Test Automation Pyramid



UI Tests (Few)

Slow, brittle, and expensive. Test the user interface and end-to-end flows. Use sparingly.



API / Integration Tests (More)

Faster and more stable. Test the "middle" layer of the application without loading a UI.



Unit Tests (Many)

The foundation. Extremely fast, stable, and check small, isolated pieces of code.

Pros & Cons of Automation

+ Pros

- **Speed:** Runs 24/7, much faster than humans.
- **Reliability:** No "tester fatigue" or human error.
- **Reusability:** Run the same test on multiple browsers.
- **Enables CI/CD:** Required for modern DevOps.

- Cons

- **Cost:** High initial setup and tool costs.
- **Skills:** Requires skilled automation engineers.
- **Maintenance:** Tests break when the UI changes.
- **"Blindness":** Cannot find new bugs, only checks what it's told.

Automation Tool: Selenium WebDriver

-  The most popular open-source tool for automating web browsers.
-  Acts as a "robot" user: it can click, type, scroll, and read text from a web page.
-  Supports many languages (Java, Python, C#, JavaScript), allowing testers to write scripts.
-  Used primarily for **Functional, Regression, and End-to-End** testing on the UI.

Part 5

Complex Case Study: "AgileBank"

Case Study: "AgileBank" E-Commerce App



System: A large e-commerce application ("AgileBank").



SDLC: Agile.



Sprint Goal: Implement a new Credit Card Payment Gateway for the checkout page.



Testing Approach: Full-stack testing across the Test Pyramid.

Case Study: Level 1 - Unit Testing

Unit Test (White Box)

A **developer** writes this test (e.g., in JUnit) to check a single function.

Goal: Does the `isValid(card)` function correctly identify an expired card?

```
@Test
public void testExpiredCard() {
    // Arrange
    PaymentGateway gateway =
        new PaymentGateway();
    CreditCard expiredCard =
        new CreditCard("1234...", "10/2023");

    // Act
    boolean isValid =
        gateway.isValid(expiredCard);

    // Assert
    Assert.assertFalse(isValid);
}
```


Case Study: Level 2 - Integration Testing

Integration Test (Grey Box)

A developer or QA writes this API test (e.g., with RestAssured).

Goal: Can our app successfully *connect* to the 3rd-party gateway and get a valid response?

```
@Test
public void testGatewayConnection() {
    // Arrange: Build the JSON payload
    String jsonBody = "{ 'apiKey': '...' }";

    // Act: Call the real API endpoint
    Response response = RestAssured
        .given()
        .body(jsonBody)
        .post("https://api.gateway.com/v1/auth");

    // Assert: Check the status code
    Assert.assertEquals(200,
        response.getStatusCode());
}
```


Case Study: Level 3 - Functional UI Testing

UI Test (Black Box)

The QA Automation Engineer writes this Selenium test.

Goal: Can a user complete a full checkout in the browser? (This is also a Regression Test).

```
@Test
public void testFullCheckoutFlow() {
    // Act 1: Login and Add to Cart
    driver.get("https://agilebank.com/login");
    driver.findElement(By.id("user"))...
    driver.findElement(By.id("product-1")).click();
    driver.findElement(By.id("checkout-btn")).click();

    // Act 2: Fill in Payment Gateway
    driver.findElement(By.id("cc-name"))...
    driver.findElement(By.id("cc-number"))...
    driver.findElement(By.id("submit-payment")).click();

    // Assert: Check for success
    WebElement successMsg =
        driver.findElement(By.id("order-success"));
    Assert.assertTrue(successMsg.isDisplayed());
}
```


Case Study: Non-Functional Testing



Performance

Use **JMeter** to run a Load Test of 500 users hitting the `/submit-payment` API endpoint to ensure it doesn't slow down.



Security

Use **OWASP ZAP** to scan the new payment fields for XSS or SQL Injection vulnerabilities.



Usability

A **Manual Tester** does exploratory testing and finds the "CVV" field label is confusing, reporting a usability bug.

Case Study: Final Step - UAT

*The **Product Owner** and a member of the **Finance Team** are given access to the Staging environment. They run their own test orders and confirm the feature meets the business requirements and that payments are correctly logged. They give the final "Go" for release.*

Part 6

The Bug Life Cycle

What Happens After Finding a Bug?

1. Report the Bug

The tester creates a bug report in a tracking tool (like Jira). A good report includes:

- Clear, concise title.
- Steps to reproduce (the "recipe").
- Expected Result vs. Actual Result.
- Severity (Impact) & Priority (Urgency).
- Screenshots, logs, and environment details.

2. Triage the Bug

The Product Owner and/or Lead Developer review the bug.

- Is it a valid bug?
- Is it a duplicate?
- What is its true priority?
- Who should fix it? (Assign)

Tracking the Bug: The Life Cycle States

-  **New:** The bug is reported and waiting for triage.
-  **Assigned / In Progress:** A developer is actively working on a fix.
-  **Ready for QA / In Test:** The developer has fixed the bug and it's ready for the *original tester* to verify.
-  **Re-Opened:** The tester finds the bug is *not* fixed and sends it back to the developer.
-  **Closed / Done:** The tester confirms the fix works and the bug is resolved.

Questions?

Thank You